



CEN415 – Project

Team Members

Abdulrahman Mansour 443106551

Hassan Nahhas 443102095

Ahmad Al sharif 437102453

Mohammed Al thagafi 443102227

Introduction

In modern digital systems, arithmetic operations form the foundation of numerous computational processes across fields such as signal processing, embedded computing, cryptography, and machine learning. Among these operations, multiplication and addition are particularly critical in determining the performance, power efficiency, and throughput of processors and custom hardware accelerators.

This project addresses the design and implementation of a custom arithmetic unit capable of executing four distinct operations: (1) parallel multiplier using Carry Save Adder (CSA) and Carry Propagate Adder (CPA), (2) shift-and-add multiplier, (3) binary subtraction, and (4) scaled addition, where the sum is multiplied by a constant. Such functionality demands an optimized architecture that supports both performance and flexibility while maintaining low hardware overhead.

The arithmetic unit accepts two 16-bit inputs (A and B), processes them according to a 2-bit selector input, and generates a 32-bit output. Emphasis will be placed on building and verifying the logic, laying out the design using Magic VLSI, and simulating functionality using IRSIM. This unit aims to serve as a practical and educational VLSI design exercise, relevant to modern computation-intensive systems.

Problem Statement and Specification

Problem Statement

The performance of digital systems often depends heavily on the speed and efficiency of their arithmetic units. As computational demand increases in real-time systems, embedded processors, and accelerators, the need for compact, fast, and reliable arithmetic circuits becomes more pronounced.

This project involves the design of a multifunctional arithmetic unit that can:

- Multiply two 16-bit inputs using two different methods (parallel CSA/CPA and shift-and-add),
- Perform 16-bit binary subtraction,
- Calculate $5 \times (A + B)$ efficiently.

The goal is to implement these operations in a single hardware module controlled by a 2-bit selector. The unit must be tested using a logic simulation tool, laid out using Magic, and simulated with IRSIM to verify correct behavior.

Specifications

Input Specifications:

- A – 16-bit input (a15:a0)
- B – 16-bit input (b15:b0)
- S – 2-bit selector (s1:s0) to choose the operation
- VDD and GND (common power supply)
- Other inputs as needed (e.g., clock, reset for shift operations)

Outputs:

- Y – 32-bit output (y31:y0)

- Optional: Carry-out or overflow indicator if needed

Operation Modes:

Operation code		Operation (Y)	Description
s1	s0		
0	0	$A * B$	Multiply A by B using parallel CSA and CPA
0	1	$A * B$	Multiply A by B using shift and add.
1	0	$A - B$	Simple subtraction
1	1	$5 * (A+B)$	Multiply the addition result by 5

Logic Gates Used:

- **Inverters:** To generate AND and OR gates.
- **NAND gates:** To generate AND gates.
- **NOR gates:** To generate OR gates.
- **AND gates:** To generate partial products.
- **OR:** To generate full adders.
- **XOR gates:** to generate full and half adders.
- **Full Adders (FA):** For summing three input bits in each column during reduction.
- **Half Adders (HA):** For summing two input bits where applicable.

Performance Goals:

- **Speed:** Reduce delay in multiplication using optimized logic paths (CSA/CPA).
- **Scalability:** Ensure architecture can be scaled to wider inputs if needed.
- **Hardware Efficiency:** Optimize for gate count and area.
- **Power Optimization:** Minimize unnecessary switching and redundant logic.

Implementation Tools:

- **Logic Design Verification:** Logisim or equivalent
- **Layout Design:** Magic VLSI tool
- **Simulation:** IRSIM

Motivation

As the demand for high-speed and efficient computation grows across industries, especially in embedded and real-time systems, the need for versatile arithmetic units becomes essential.

Traditional designs often focus on individual operations such as multiplication or addition; however, integrating multiple arithmetic functions into a single unit can improve efficiency and reduce area.

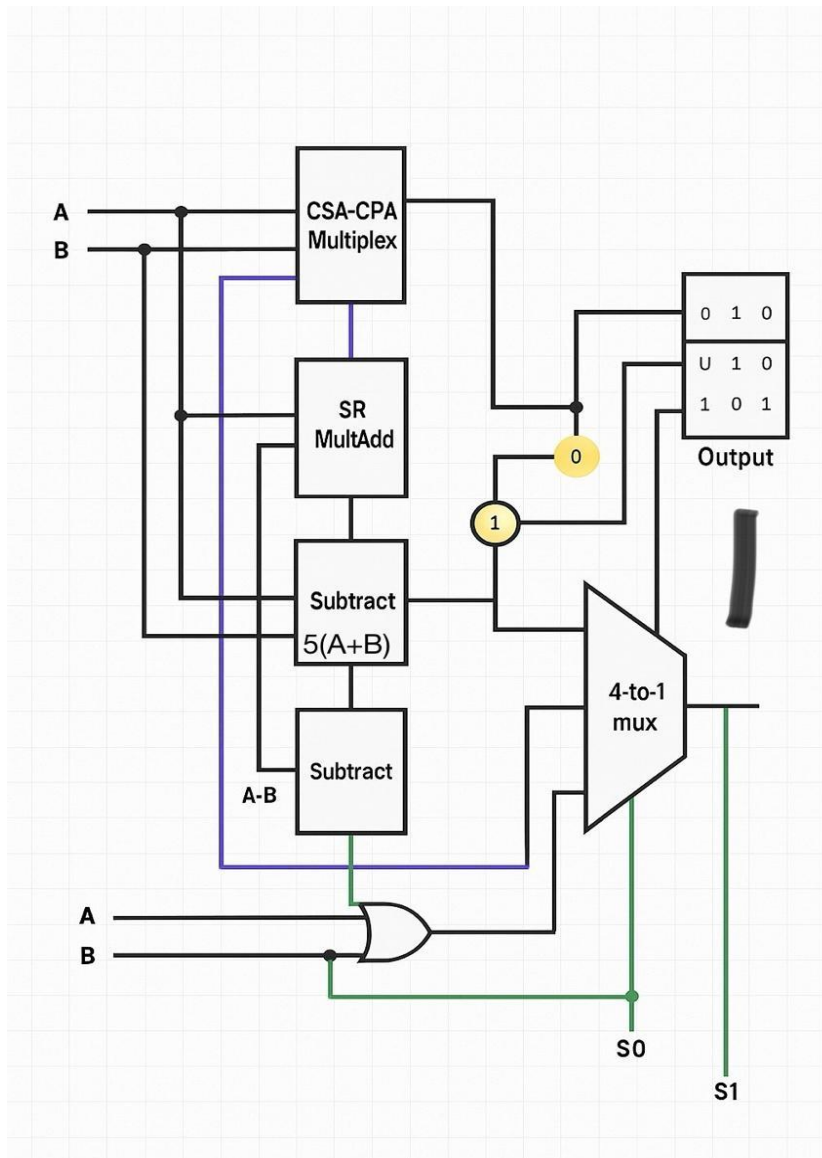
This project is motivated by the need to explore VLSI-level implementation of a compact yet powerful arithmetic unit. The

chosen operations reflect commonly used patterns in many practical applications, including signal processing, control systems, and data compression.

By implementing and simulating the circuit at both the logic and layout levels, this project offers a hands-on experience in bridging theoretical digital design and physical VLSI layout.

The goal is to build a foundational yet extensible unit that aligns with the requirements of modern, low-power, high-performance digital systems.

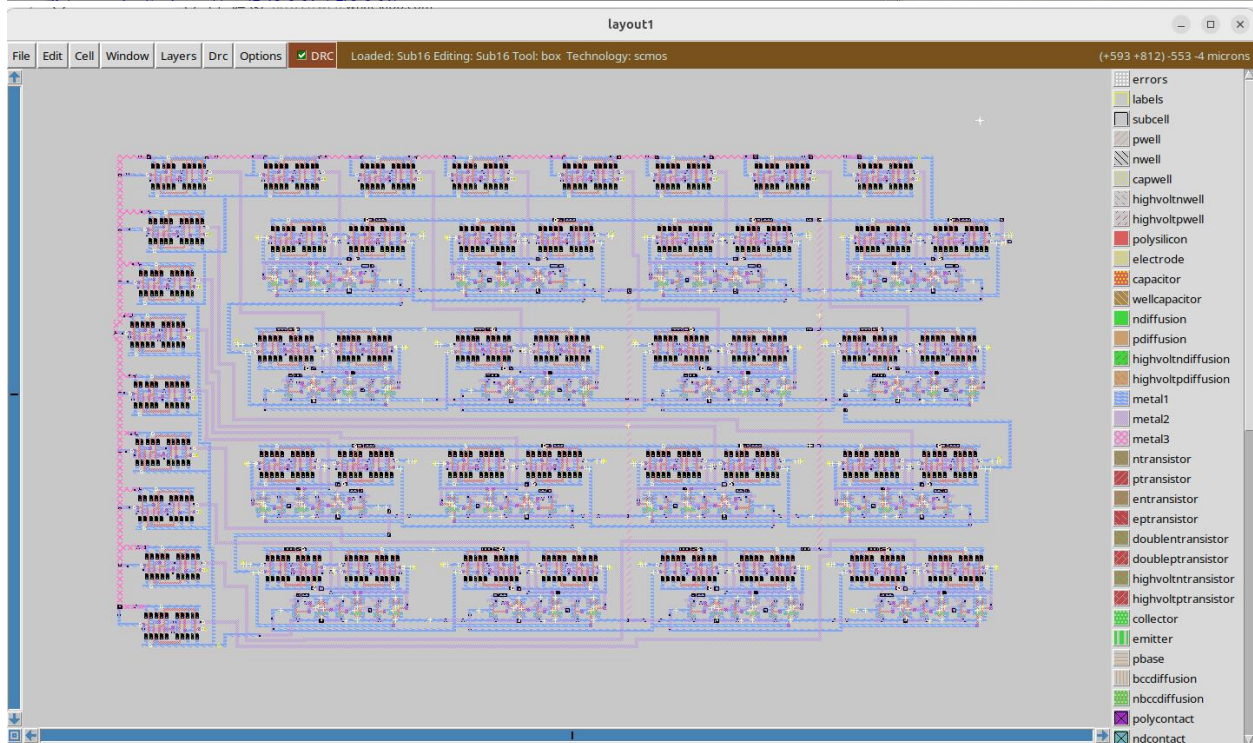
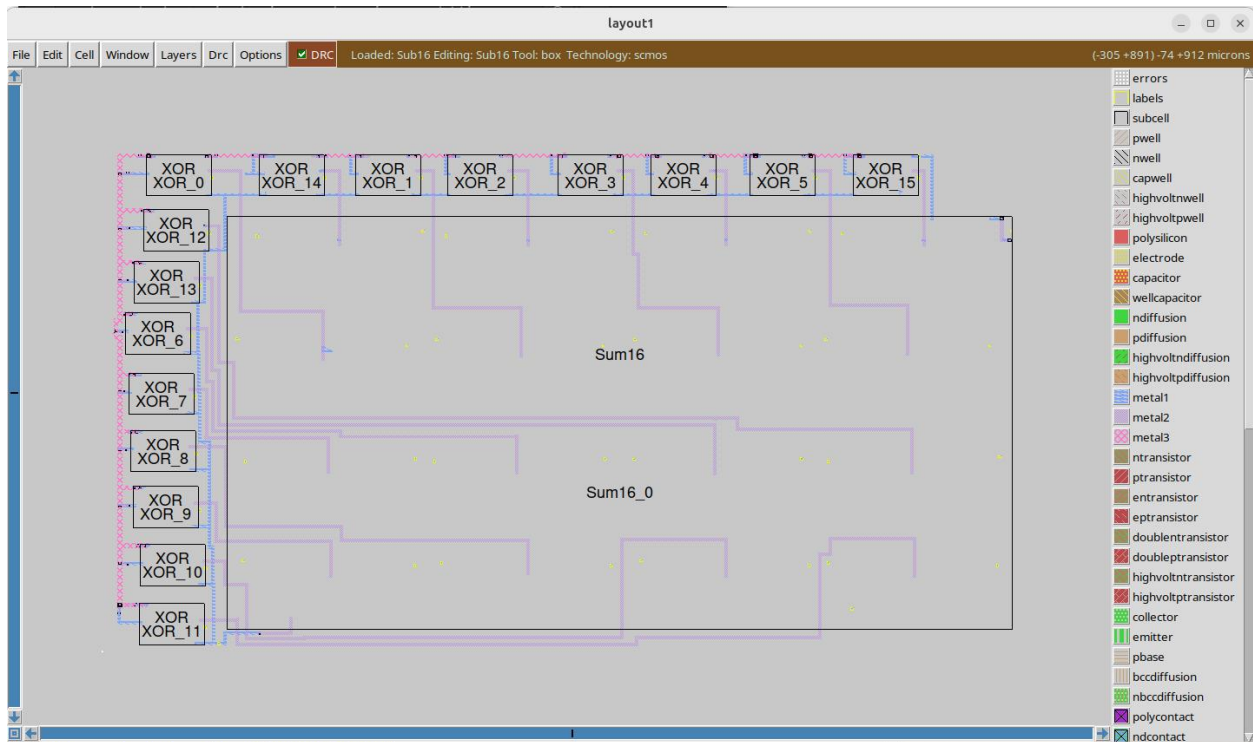
Block diagram



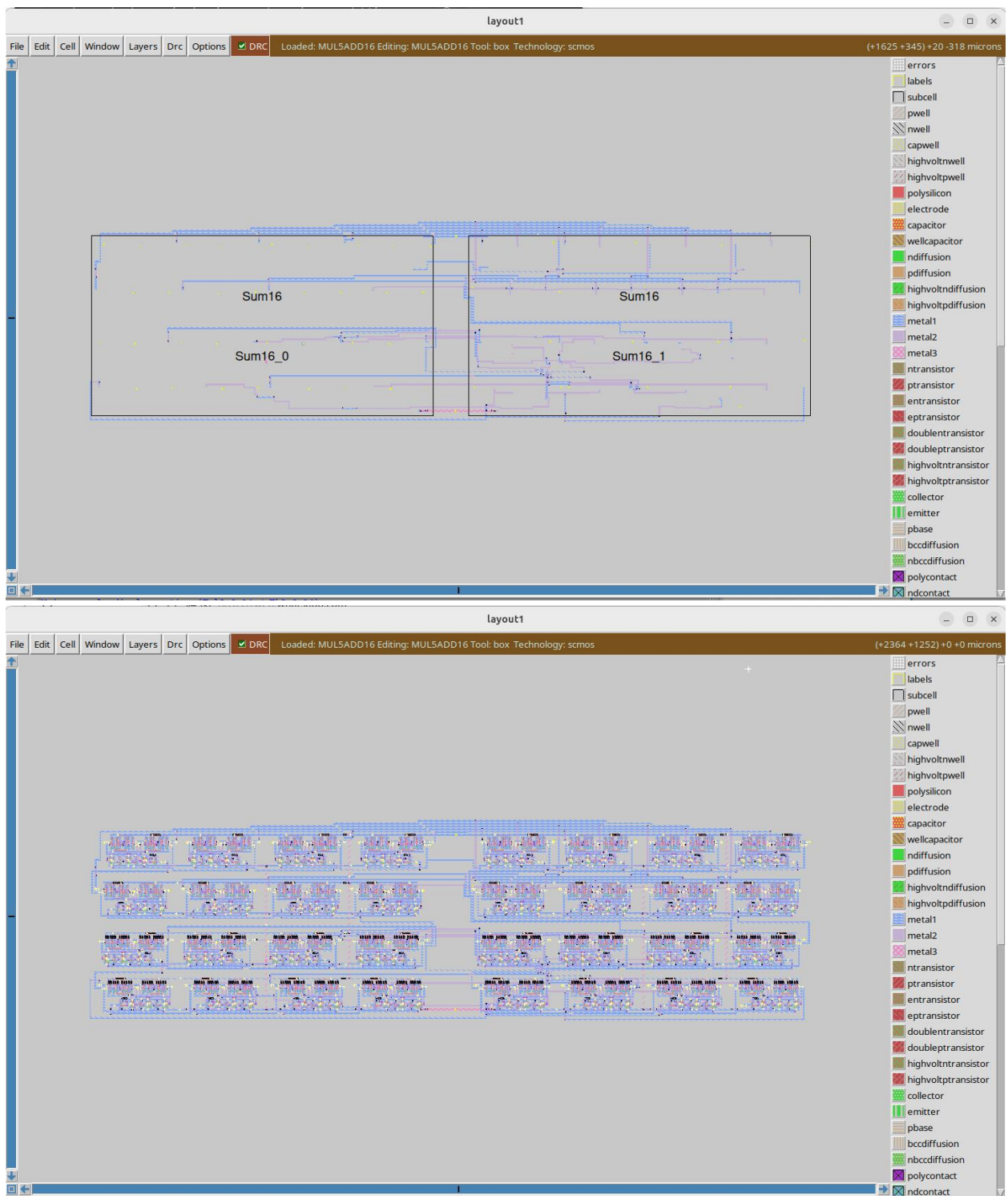
Note: We apologize, but we did not have enough time to compile all the project components into MUX. However, the components and even the MUX are ready, tested, and working perfectly.

Logic Design Soultion

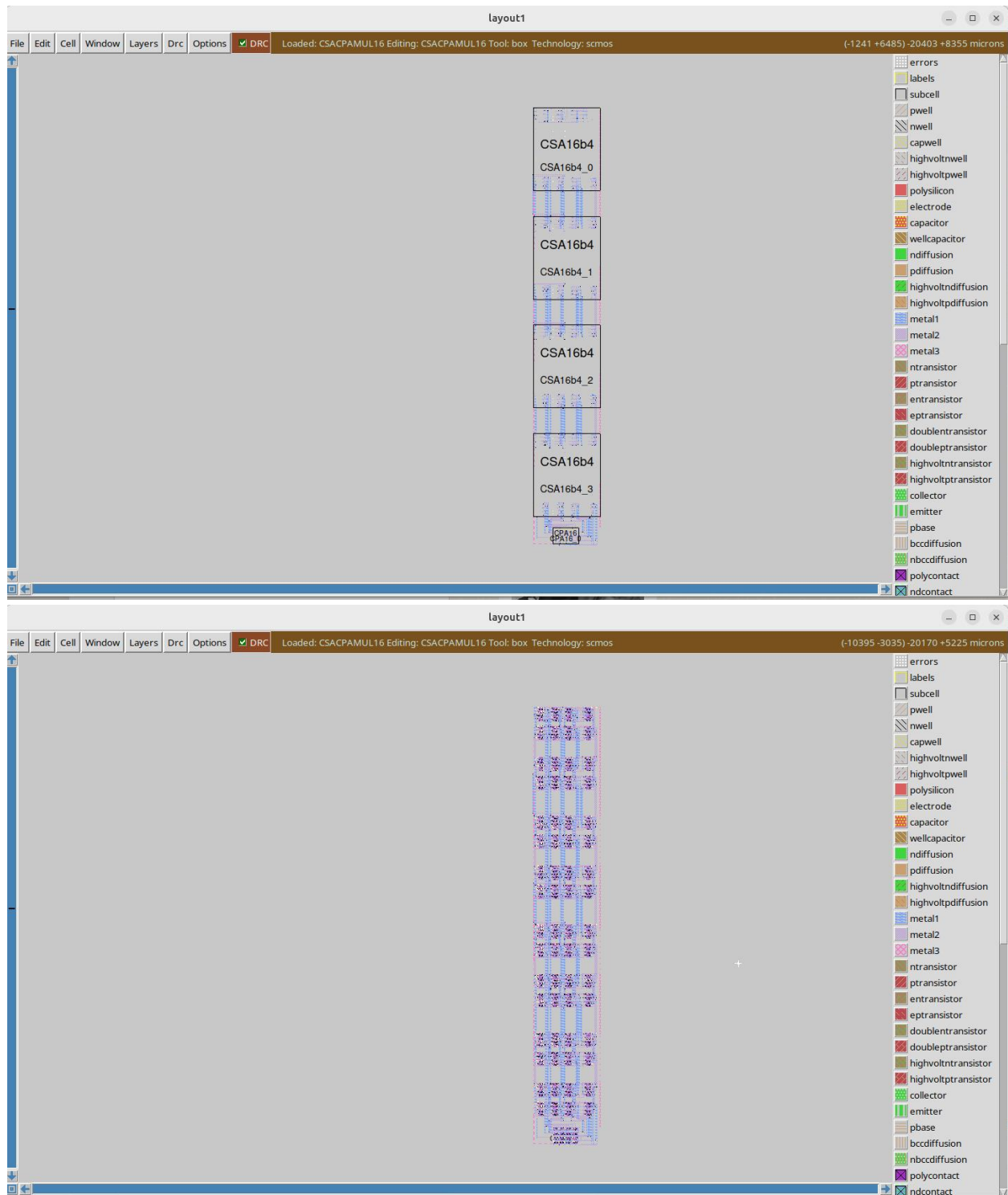
Subtract:



Add then multiply by 5:



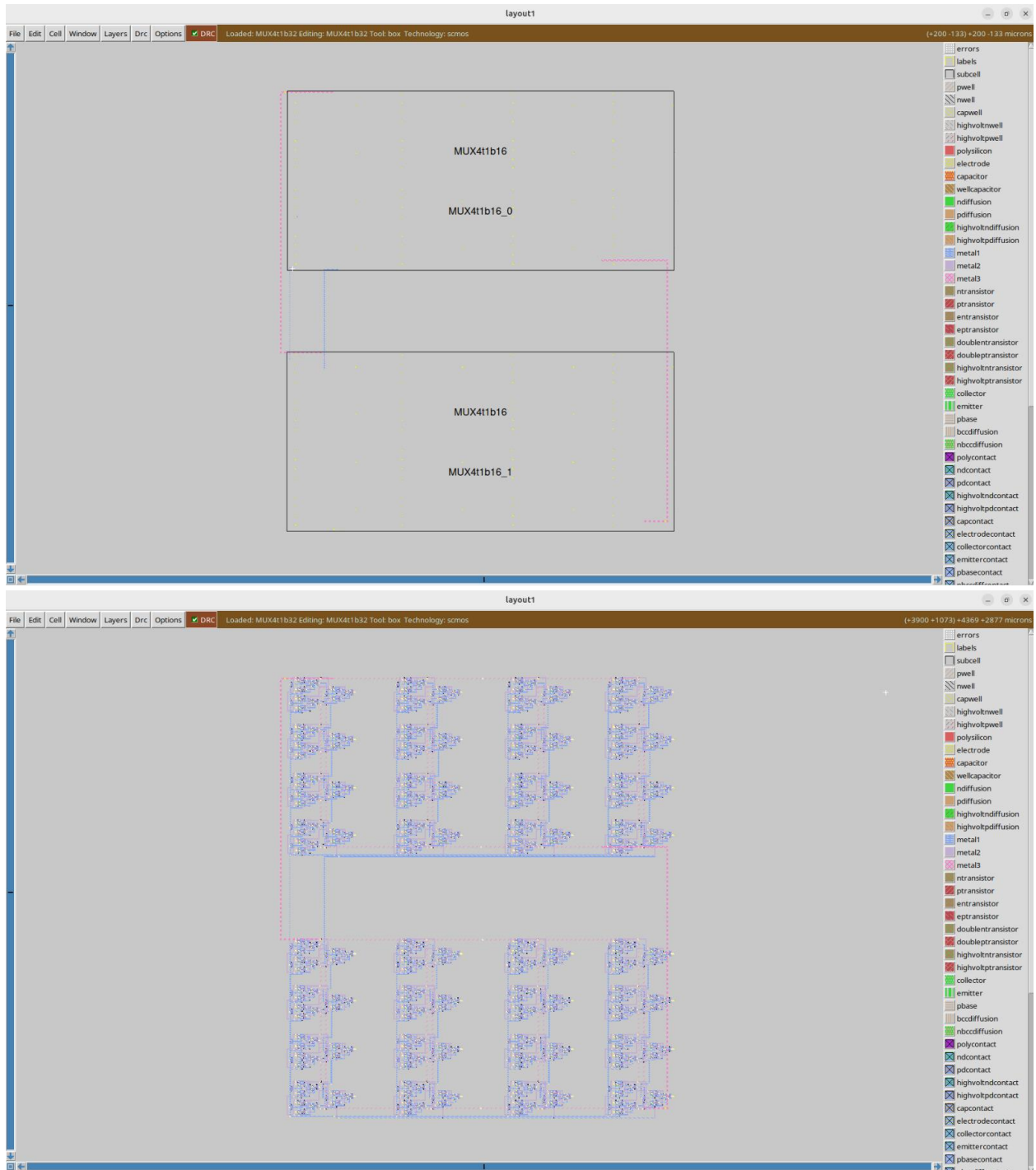
CSA and CPA MUL:



Shift Add Multiplier:

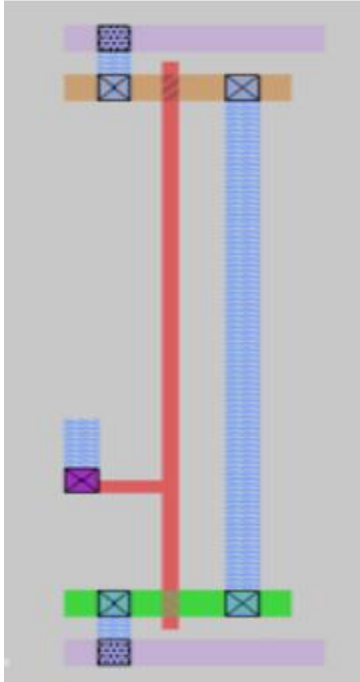


MUX 4 to 1:

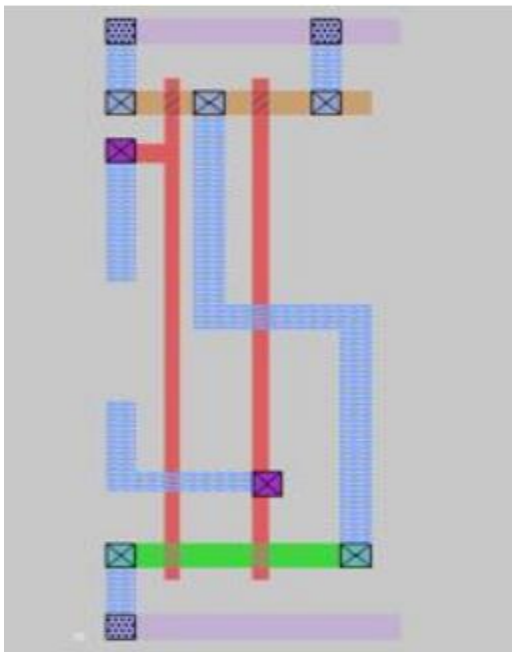


STICK DIAGRAMS

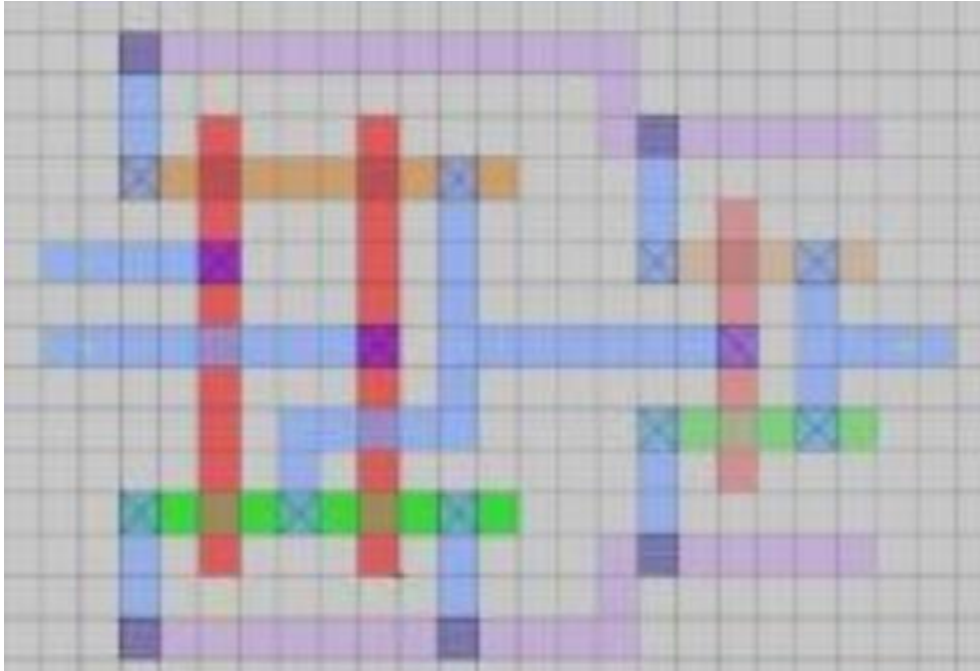
Inverter:



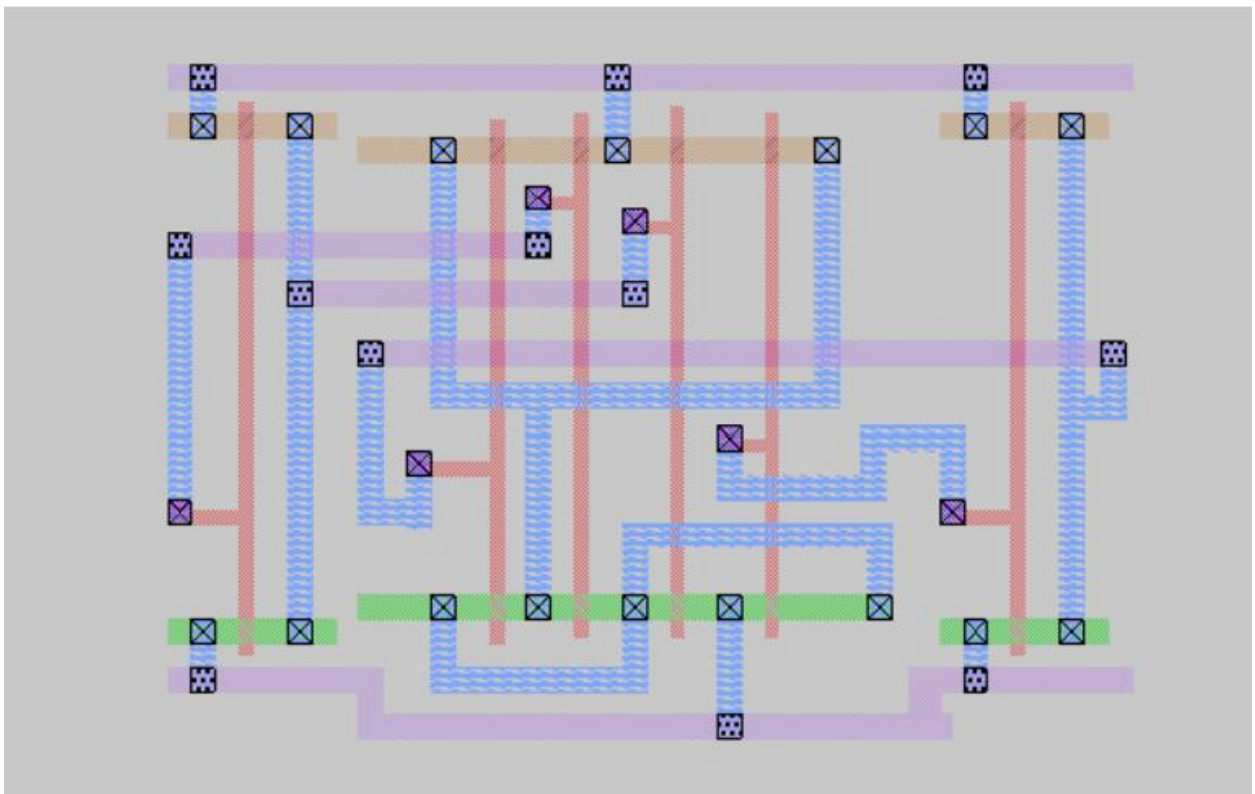
NAND:



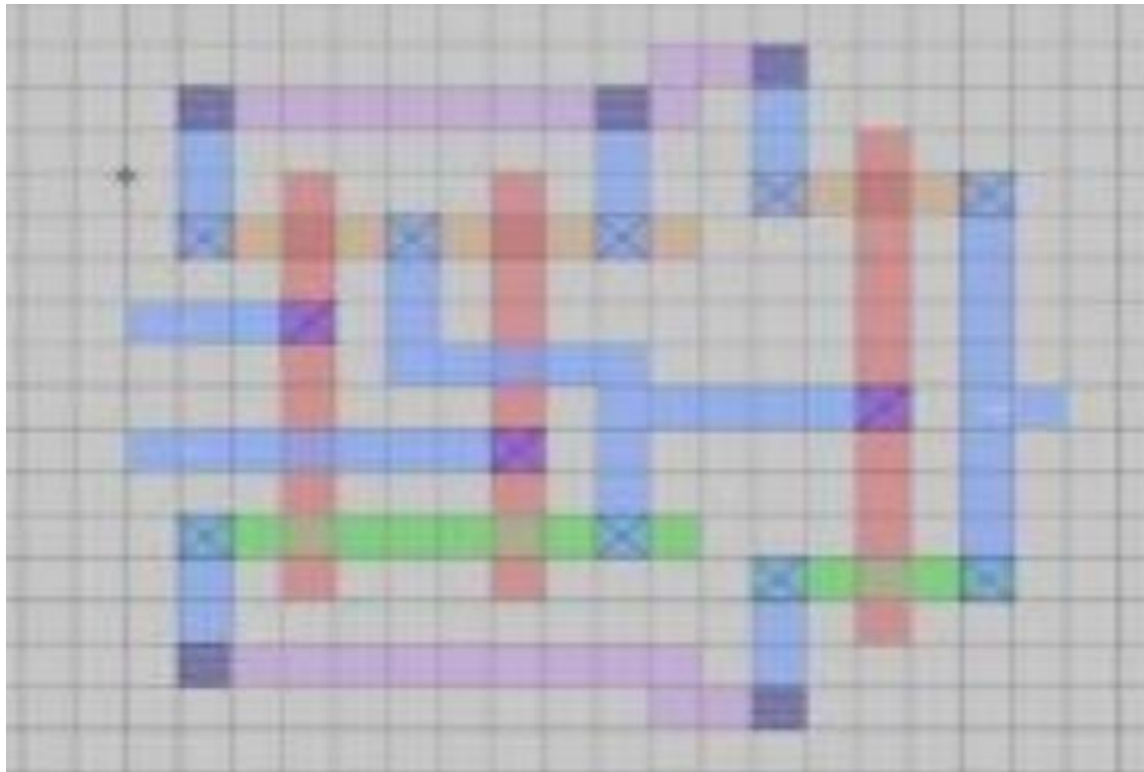
NOR:



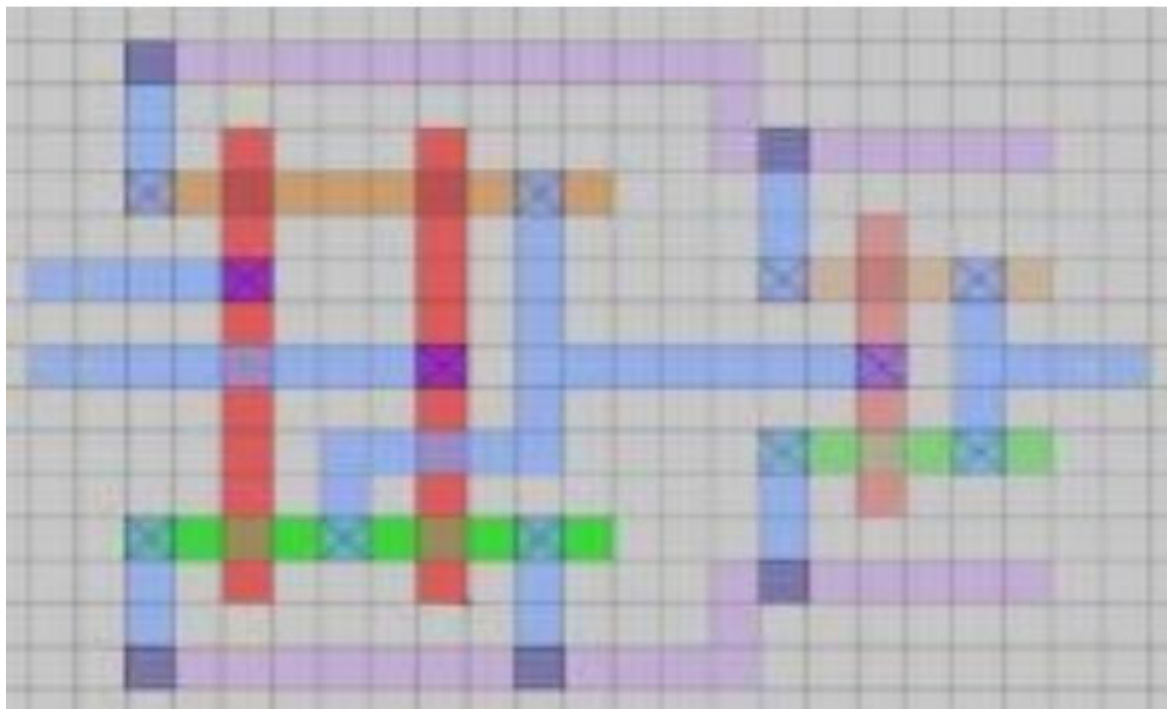
XOR:



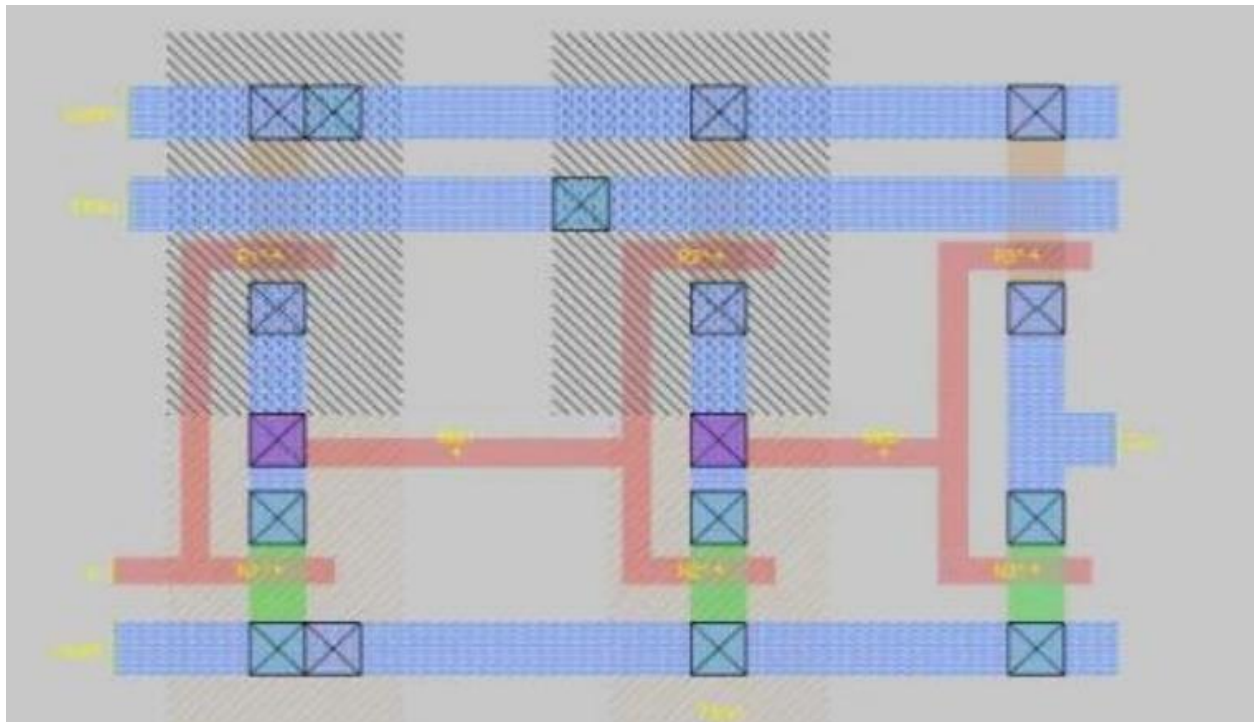
AND:



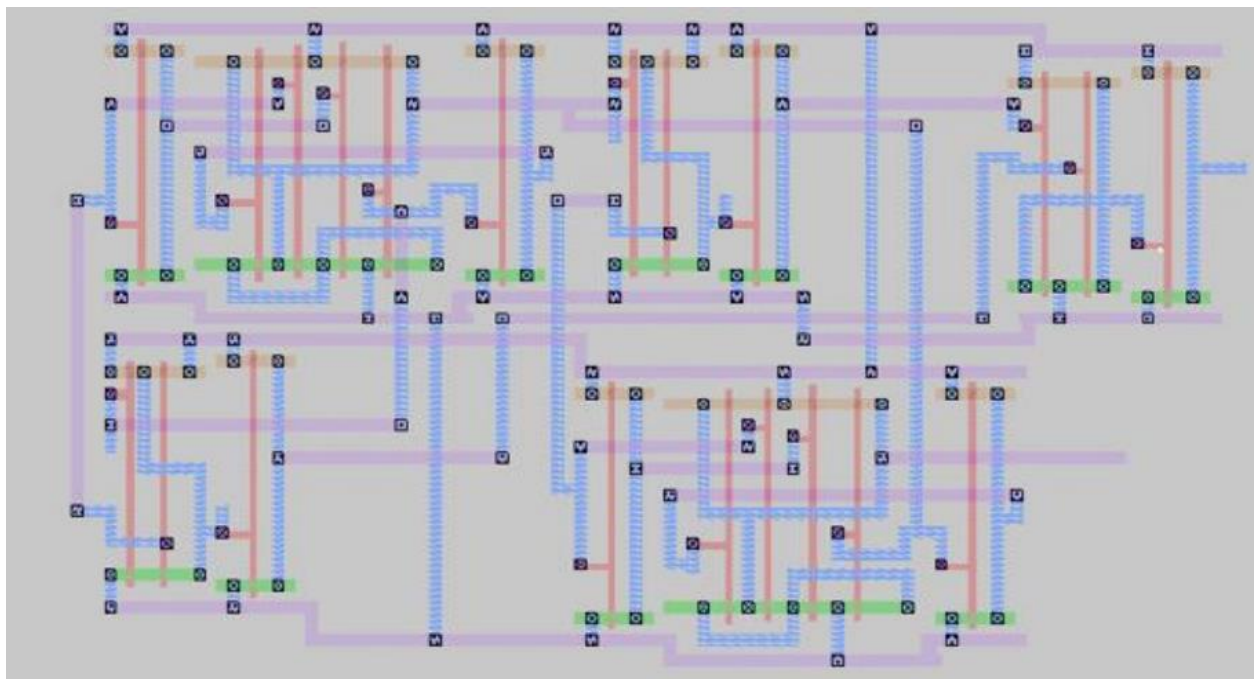
OR:



Half Adder:



Full Adder:



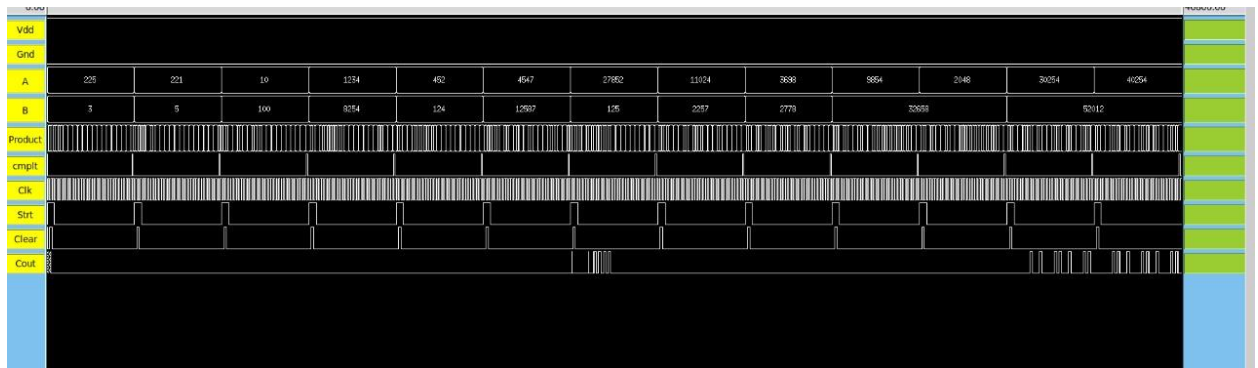
Testing Strategy and Results

IRSIM simulation Results

CSA CPA multiplication



Shift add multiplier



Subtractor



Add then multiply by 5

	0.00										550.00
Vdd											
Gnd											
A	11	3452	21	400	23	29	36	1	10	35813	
B	50	2260	131	20	7000	123	50	1	5	50	24187
Sum	396	38910	760	2190	35115	760	430	10	30	300	37706
Cout											
Sumtst0	61	9702	102	420	7023	102	06	2	5	50	59970

Multiplexer

	0.00					250.00
Vdd						
Gnd						
I0	235410			157514		
I1	63520			075424		
I2	50744			1254121		
I3	570011			150744		
S	00	01	10	11	01	
Out	235410	63520	50744	150744	075424	

Loading the Layout (Magic) and Simulation (IRSIM) Commands

Loading Magic Design

```
cd [design_directory]
```

```
magic [Design_Name]
```

Extraction and sim file generation

```
save
```

```
extract all
```

```
ext2sim
```

Running IRSIM and Loading Simulations

```
irsim scmos100.prm design.sim -testing_code.cmd
```